GUÍA EXPLICATIVA + GUÍA DE EJECUCIÓN

¿Qué construimos y cómo se usa?

Una explicación sencilla del simulador logístico, la interfaz, los servicios que trabajan por detrás y los pasos para correr la demo.

La idea en una frase

Simulamos envíos y hacemos que varias partes del sistema reaccionen a cada evento: actualizan el seguimiento, calculan alertas, registran notificaciones y generan métricas.

Preparado para entender el proyecto y presentarlo paso a paso.

EL PROYECTO, EXPLICADO SIMPLE

¿Qué es este TP?

Es una maqueta funcional de una empresa de logística. No transporta paquetes reales: genera envíos de prueba y simula lo que pasa desde que se crean hasta que se entregan, fallan o necesitan atención.

1	2	3	4
Se crea un envío	Se publican eventos	Varios servicios reaccionan	La web muestra el resultado

¿La interfaz la hicimos nosotros o ya venía hecha?

Por lo que muestra el código del TP, la interfaz de RutaSur está armada para este proyecto: tiene sus propias pantallas, textos, estilos, botones y navegación. No es un panel logístico que venga listo dentro de Next.js. **Next.js y React son las herramientas** con las que se construyó; la aplicación RutaSur es el producto del TP.

Se puede reconocer en las páginas y componentes del proyecto: torre de control, demo, analítica, tarjetas de escenarios, panel de lag y páginas de seguimiento. Los frameworks aportan la base y herramientas, como una caja de herramientas; el equipo escribe la aplicación que se ve y su comportamiento.

¿Qué problema muestra?

En un sistema real hay muchas cosas que deben pasar cuando cambia un envío. En vez de que un único programa haga todo en cadena, acá cada servicio escucha los eventos que le interesan. Así se puede mostrar procesamiento en paralelo y qué pasa cuando una parte se demora o se detiene.

Un envío es la historia

Su identificador permite seguir sus estados, intentos, alertas y reprogramaciones.

Un evento es un aviso

Por ejemplo: “entró al centro”, “salió a reparto” o “la entrega falló”.

Los servicios son equipos

Cada consumidor tiene una tarea y puede trabajar sin esperar a todos los demás.

La interfaz es la vidriera

Resume en pantallas lo que está pasando dentro del sistema.

DEBAJO DE LA INTERFAZ

El recorrido de la información

El simulador manda avisos a Kafka. Los consumidores leen esos avisos y actualizan datos o hacen su tarea. La web consulta los datos guardados en MongoDB y los presenta.



La interfaz Next.js consulta MongoDB. Los botones de la demo llaman al simulador para pedirle que publique un escenario.

¿Qué hace cada servicio?

Parte	En criollo
Simulador	Inventa envíos, estados y situaciones de prueba; los publica en Kafka.
Kafka	Es una bandeja de eventos persistente. Cada tipo de aviso viaja por un canal llamado topic.
Proyector	Escucha eventos y arma la ficha consultable del envío, las alertas y las notificaciones.
Notificaciones	Registra un aviso simulado cuando falla una entrega.
Reprogramación	Decide si reintenta otro día, retiene el envío o lo devuelve al comercio.
Monitor SLA	Compara la promesa de entrega con el avance y crea alertas si hay riesgo.
Analítica	Acumula indicadores como entregas en fecha, primer intento y causas de fallo.
MongoDB	Guarda los datos preparados que la web necesita mostrar.
Next.js + React	Construyen y sirven las páginas web de RutaSur.

Los canales principales (topics)

Los nombres son técnicos, pero describen qué circula por cada canal: `envios.estados`, `envios.promesas`, `envios.reprogramaciones`, `alertas.sla`, `notificaciones.enviadas` y `envios.estados.dlq`.

> **¿Por qué varios consumidores?** Como varios equipos que reciben una copia del aviso: uno puede actualizar el envío, otro avisar al destinatario y otro calcular estadísticas. Si uno se frena, los otros pueden seguir.

LO QUE SE VE EN EL NAVEGADOR

Las pantallas de RutaSur

La interfaz organiza la información para que se pueda entender y demostrar el flujo sin tener que leer directamente los mensajes de Kafka o consultar la base a mano.

Pantalla	Qué muestra	Cómo entrar
Torre de control	Contadores por estado, últimos movimientos y alertas SLA abiertas.	`/`
Panel de demo	Botones para lanzar casos preparados y un panel de lag de consumidores.	`/demo`
Analítica	Indicadores, motivos de fallo y filtros por zona, centro y fecha.	`/analytics`
Detalle del envío	Información e historial del envío elegido.	`/shipments/ID`
Tracking	Vista para seguir un envío por su identificador.	`/tracking/ID`

¿Qué significa “EN VIVO”?

La aplicación actualiza algunas pantallas periódicamente para reflejar cambios recientes. Usa SSE (eventos enviados desde el servidor al navegador) para avisar sobre actualizaciones, y el panel de lag vuelve a consultar cada cinco segundos.

¿Qué es el lag?

Es la cantidad de mensajes que le faltan procesar a un consumidor. Si detenemos uno y el simulador sigue publicando, su lag crece. Al volver a iniciarlo, puede continuar desde donde quedó. Es una forma visual de mostrar que los consumidores están desacoplados.

Los botones de escenarios

Los botones no cambian los datos mágicamente en el navegador. Envían un pedido a la API del sitio; esta llama al simulador, y el simulador publica los eventos. Después los consumidores procesan esos eventos y la interfaz muestra el resultado.

> **Respuesta corta para la presentación**
>
> “La interfaz de RutaSur la construimos para este TP con React y Next.js. Esas tecnologías nos dan la estructura para crear páginas; no nos entregan ya hecha esta torre de control, este panel de demo ni estas pantallas de envíos.”

SIMULAR, PROBAR Y VOLVER A PROCESAR

¿Qué se puede repetir?

Interpreto “lo iterable” como la parte que se puede volver a ejecutar o procesar. En este TP hay varias maneras de hacerlo:

Recurso	Para qué sirve
Simulador continuo	Publica envíos automáticamente. Por defecto genera 20 por minuto y usa un reloj acelerado 120 veces.
Escenarios de demo	Repite casos concretos: entrega exitosa, ausencia, demora con alerta SLA, tercer fallo, ráfaga o evento inválido.
Ráfaga	Publica 500 envíos para observar acumulación y recuperación de los consumidores.
Replay	Lee el historial de estados de Kafka desde el comienzo con un grupo nuevo y recalcula el tiempo medio en el centro.
Rebuild	Vuelve a construir las vistas derivadas de MongoDB desde el log de Kafka. El log no se borra.

Casos que conviene mostrar

1. **Envío feliz:** seguir el recorrido hasta “Entregado” y abrir el tracking.
2. **Entrega fallida:** mostrar la causa, la notificación y la decisión de reprogramar.
3. **Envío demorado:** observar la alerta que avisa que la promesa corre riesgo.
4. **Tercer fallo:** ver que el paquete vuelve al comercio.
5. **Ráfaga:** observar los mensajes pendientes y cómo los consumidores los procesan.
6. **Evento inválido:** mostrar que un evento malformado va a la DLQ, una bandeja de errores.

> ### Ojo con “replay” y “rebuild”
>
> Replay recalcula un indicador adicional sin reiniciar a los otros consumidores. Rebuild es una operación más grande: detener consumidores y simulador, vaciar proyecciones y reconstruirlas desde el historial. Para una demo normal no hace falta ejecutarlo.

¿Y si un consumidor se detiene?

Los eventos quedan en Kafka. Los demás consumidores siguen trabajando. Al reiniciar el que se detuvo, procesa lo que le faltaba. Esa independencia es una de las ideas principales del TP.

DICCIONARIO EN CRIOLLO

¿Qué usamos y para qué?

Tecnología o palabra	Explicación sencilla
JavaScript / Node.js	El lenguaje y el entorno que ejecutan el simulador y los servicios del servidor.
React	Permite construir la interfaz con piezas reutilizables, como tarjetas, botones y paneles.
Next.js	Organiza páginas, rutas y APIs de la aplicación web y la ejecuta.
Kafka	Guarda y reparte eventos. Es como una fila de avisos a la que varios equipos pueden suscribirse.
MongoDB	Base de datos donde se guardan los envíos proyectados, las alertas, notificaciones y métricas.
Docker	Empaqueta cada servicio para que se ejecute en un contenedor con su entorno.
Docker Compose	Levanta y conecta varios contenedores con un solo comando.
Zod	Valida que los eventos tengan los datos esperados antes de procesarlos.
Topic	Canal de Kafka para un tipo de evento.
Consumidor	Proceso que lee eventos de Kafka y realiza una tarea.
Offset	La “marca de hasta dónde leí” de un consumidor.
Proyección	Una vista preparada para consultar rápido, por ejemplo el estado actual y el historial de un envío.
DLQ	Bandeja aparte para mensajes que no se pudieron procesar correctamente.
SLA	Compromiso de tiempo de entrega; el monitor alerta cuando hay riesgo de incumplirlo.
Idempotencia	Si un evento se procesa de nuevo, se evita duplicar el efecto en la base de datos.

Por qué no se conectan todos directamente entre sí

Kafka funciona como intermediario: el simulador publica y cada consumidor se ocupa de lo suyo. Así, para sumar o detener un consumidor no hace falta cambiar el simulador ni frenar a todos los demás.

El simulador usa datos de ejemplo, una semilla para hacer reproducible la generación y probabilidades configurables. Es una demostración académica: no integra un servicio de mensajería real ni transportistas reales.

GUÍA DE EJECUCIÓN EN WINDOWS

Pasos para correr todo el TP

1. **Abrí Docker Desktop** y esperá a que indique que está iniciado.
2. **Abrí PowerShell** y entrá a la carpeta del proyecto:

```
cd "C:\Users\Briisa\TP7-Prog. Avanzada"
```

3. **Levantá los servicios:**

```
docker compose up --build -d
```

La primera vez descarga imágenes y construye la aplicación; puede tardar. El modificador `-d` deja los contenedores corriendo en segundo plano.

4. **Comprobá el estado:**

```
docker compose ps -a
```

MongoDB y Kafka deben aparecer saludables; los servicios principales, como `web`, `simulator` y los consumidores, deben figurar activos.

5. **Abrí el navegador:** `http://localhost:3100`. Desde allí, entrá a Demo o Analítica. La salud de la API se comprueba en `http://localhost:3100/api/health`.
6. **En Demo, ejecutá los escenarios** y observá cómo se actualizan la torre de control, las alertas, el tracking y la analítica.

Comandos útiles

```
# Ver mensajes de los servicios; Ctrl+C sale de la vista
docker compose logs -f

# Correr pruebas locales en PowerShell de Windows
npm.cmd test

# Apagar contenedores sin borrar los datos guardados
docker compose down
```

> **Importante: no borres el volumen sin querer**
>
> `docker compose down` conserva los datos. Evitá `docker compose down -v` salvo que quieras borrar también los volúmenes, incluidos los datos persistentes de MongoDB.

Si vuelve el error HTTP 500 de Docker, reiniciá Docker Desktop, esperá a que el motor Linux termine de arrancar y repetí `docker compose ps -a`.